LABORATORIES

LAB-1 (RECAP)

1. Learn how to use GIT
   - Install: https://git-scm.com/downloads
   - Documentation: https://git-scm.com/docs/
   - Learn simple command:
     1. commit --> https://git-scm.com/docs/git-commit
     2. clone --> https://git-scm.com/docs/git-clone
     3. push --> https://git-scm.com/docs/git-push
     4. pull --> https://git-scm.com/docs/git-pull
     5. status --> https://git-scm.com/docs/git-status
     6. add --> https://git-scm.com/docs/git-add
     7. log --> https://git-scm.com/docs/git-log
     8. config --> https://git-scm.com/docs/git-config
   - Each student will create his/hers private GITHUB repository where his/her teaching assistant will be set as a collaborator for that repo.
   - All of the operations from now on (lab exams, lab homework will be done via that repo (or multiple repos)).
2. Write a program in C-Language that open the file "in.txt", and prints the sum of the numbers that are found on each line of the file in.txt. To open the file use **fopen** API. Write your own function that converts a string to a number (similar cu **atoi** API). To print something to the screen, use the **printf** API.
   Example: let's consider the following "ini.txt" file:
   123
   198698
   5009
   983279
   The program will print to the screen 1187109.

3. Read a sentence from the input using **scanf** API. Then sort split it into words and write to the screen (using the **printf** API) the words sorted (from the longest one to the shortest one).If two words have the same length - they will be sorted out alphabetically. We consider that each word is separated from another one using space (one or multiple).
   Example: let's consider the following sentence: "*I went to the library yesterday*". The program will print the following to the screen:
   yesterday
   library
   went
   the
   to
   I

4. Lets consider the following code:

```
#include <iostream>
using namespace std;

bool isPrime(int n)
{
    for (int tr = 2; tr < n / ......; tr++)
        if ((n % ...... ) == 0)
            return ......;
    return true;
}
int main()
{
    int n;
    std::cout << "Enter a number:";
    std::cin >> ......;
    if (isPrime(n))
        std::cout << n << " is prime !";
    else
        std::cout << n << " is NOT prime !";
    return 0;
}
```

Fill in the missing code (dots: .....) to make this program work as expected.

LAB-2 (SIMPLE CLASSES)

1. Write a class in C++ that has the following definition:

```
class NumberList
{
    int numbers[10];
    int count;
public:
    void Init();          // count will be 0
    bool Add(int x);      // adds X to the numbers list and increase the data member count.
                          // if count is bigger or equal to 10, the function will return false
    void Sort();          // will sort the numbers vector
    void Print();         // will print the current vector
}
```

Organize the code in the following way:

- a header file called **NumberList.h**
- a cpp file called **NumberList.cpp** that contains the source code for class **NumberList**
- a main file called **main.cpp** that contains the main function and has an example on how to use **NumberList**. The example must include using all methods from the class.
- make sure that precompile headers are NOT used (for Visual Studio setup).

2. Design a C++ class (following the previous problem) that reflects the properties of a student. The class should include the following:
   - methods to set and get the name of the student
   - methods to set and get the grade for mathematics (a grade must be a float value from 1 to 10)
   - methods to set and get the grade for English (a grade must be a float value from 1 to 10)
   - methods to set and get the grade for History (a grade must be a float value from 1 to 10)
   - a method that retrieves the average grade
   - 5 global functions that compares two students in terms of name, grades, average. If two students are equal the return value of such a function will be 0. If the first student is bigger than the second one, the return value will be 1, otherwise -1.

Make sure that you have the following:

- a header file for the class
- a cpp file for the methods specific to the class
- a header file for the global functions
- a cpp file for the global functions implementation
- a main.cpp file that shows how the methods and global functions can be used.

LAB-3 (METHODS)

1. Write a class in C++ that has the following definition:

```
class Math
{
public:
    static int Add(int,int);
    static int Add(int,int,int);
    static int Add(double,double);
    static int Add(double,double,double);
    static int Mul(int,int);
    static int Mul(int,int,int);
    static int Mul(double,double);
    static int Mul(double,double,double);
    static int Add(int count,...); // sums up a list of integers
    static char* Add(const char *, const char *)
}
```

Organize the code in the following way:

- a header file called **Math.h**
- a cpp file called **Math.cpp** that contains the source code for class **Math**
- a main file called **main.cpp** that contains the main function and has an example on how to use **Math**. The example must include using all methods from the class.
- for the variadic method use pointers or **va_start** / **va_end** macros.
- Add(const char *, const char *) will allocate memory that can add the two existing string. If one of the strings is nullptr, this function will return nullptr;

2. Write a class in C++ that has the following definition:

```
class Canvas
{
    // add private data members
public:
    Canvas(int width,int height);
    void DrawCircle(int x, int y, int ray, char ch);
    void FillCircle(int x, int y, int ray, char ch);
    void DrawRect(int left, int top, int right, int bottom, char ch);
    void FillRect(int left, int top, int right, int bottom, char ch);
    void SetPoint(int x, int y, char ch);
    void DrawLine(int x1, int y1, int x2, int y2, char ch);
    void Print(); // shows what was printed
    void Clear(); // clears the canvas
}
```

Organize the code in the following way:

- a header file called **Canvas.h**
- a cpp file called **Canvas.cpp** that contains the source code for class **Canvas**
- a main file called **main.cpp** that contains the main function and has an example on how to use **Canvas**. The example must include using all methods from the class.
- use a matrix of type char for the canvas. A pixel is a cell in the matrix. A pixel is considered empty if it contains the space (value 32) character.

For the DrawLine algorithm use **Braseham** algorithm.

LAB-4 (CLASSES)

Write a class in C++ that has the following definition:

```
class Sort
{
    // add data members
public:
    // add constuctors
    void InsertSort(bool ascendent=false);
    void QuickSort(bool ascendent=false);
    void BubbleSort(bool ascendent=false);
    void Print();
    int  GetElementsCount();
    int  GetElementFromIndex(int index);
}
```

Organize the code in the following way:

- a header file called **Sort.h**
- a .cpp file called **Sort.cpp** that contains the source code for class **Sort**
- a main file called **main.cpp** that contains the main function and has an example on how to use **Sort**. The example must include using all methods from the class.
- add several constructors that will allow the following:
  1. create the list that needs to be sorted out of random values within a specific interval (min, max). The constructor will receive 3 parameters (the number of elements in the list, minimum value, maximum value).
  2. create the list that needs to be sorted from an initialization list
  3. create the list that needs to be sorted from an existing vector (the constructor will have two parameters - one being the vector, the other one being the number of elements from the vector)
  4. create the list that needs to be sorted using variadic parameters (use va_args for this)
  5. create the list that needs to be sorted from a string (e.g. "10,40,100,5,70" -> each number is separated from the rest of the number with a comma). It is assumed that the string is correctly written (no space, only numerical characters and commas)

OBS: By "list" in this context we refer to a vector or any form of container that stores elements (it could be a double-linked-list, a heap vector or an array).

LAB-5 (OPERATORS)

1. Write a class in C++ that has the following definition:

```
class Number
{
   // add data members
public:
   Number(const char * value, int base); // where base is between 2 and 16
   ~Number();

   // add operators and copy/move constructor

   void SwitchBase(int newBase);
   void Print();
   int  GetDigitsCount(); // returns the number of digits for the current number
   int  GetBase(); // returns the current base
}
```

Organize the code in the following way:

- a header file called **Number.h**
- a cpp file called **Number.cpp** that contains the source code for class **Number**
- a main file called **main.cpp** that contains the main function and has an example on how to use **Number**. The example must include using all methods from the class.
- add the following operators: addition, subtraction, index operator, relation operators (> , < , >= , <=, ==, etc)
- add copy & move constructors and move assignment operator
- when performing operations with two **Number** object that have a different base, the result (except for the relation and index operators) will have the biggest base of the two Number instances.
- for addition and subtraction use **friend** functions
- implement the -- operator with the following syntax: if used in a prefix form it will remove the first (most significant digit) from the number; if used in a post-fix form it will remove the last (least significant) digit from the number;

Example:

```
int main()
{
   Number n1("10110010",2);
   Number n2("734",8);
   Number n3("FF",16);

   printf("n1 has %d digits and it is written in base %d\n",n1.GetDigitsCount(),n1.GetBase());
   for (int tr=0;tr<n1.GetDigitsCount();tr++)
   {
      printf("n1[%d]=%c\n",tr,n1[tr]);
   }
   n1.Print();
   n2.Print();
   n1 = (n2+n3-n1)+n1; // after this n1 will be in base 16
   n1.SwitchBase(2);
   n1.Print();

   if (n1>n2) printf("n1 is bigger than n2\n"); else printf("n2 is bigger than n1\n");

   Number n4=12345; // n4 will be in base 10
   n1 = 255; // n1 will be 11111111 (value 255 from base 10 in base 2)
   n4 += n1;
   n4.Print();

   n4 = "13579"; // n4 mentains its base (10) and will be 13579
   n4.Print();
   --n4; // the first digit from n4 will be remove ==> n4 becomes 3579
   n4.Print();
   n4--; // the last digit from n4 will be remove ==> n4 becomes 357
   n4.Print();

   return 0;
}
```

LAB-6 (INHERITANCE)

Write the following classes in C++:

- An abstract class **Car** that reflects some properties on a car (fuel capacity, fuel consumption, average speed on different conditions (Rain, Sunny, Snow)).
- Derive from this class several type of Cars with different characteristics for fuel capacity, fuel consumption, etc.
- For weather condition create an ***enum*** object.
- Create a class circuit that will test how different cars behave in a race depending on the length of the circuit and the race weather condition. The circuit will use polymorphism to simulate the race conditions (e.g. will have an array of objects of type ***Car****).
- Use the following **main** snippet to show how this ensemble works:

```
int main()
{
   Circuit c;
   c.SetLength(100);
   c.SetWeather(Weather::Rain);
   c.AddCar(new Dacia());
   c.AddCar(new Toyota());
   c.AddCar(new Mercedes());
   c.AddCar(new Ford());
   c.AddCar(new Mazda());
   c.Race();
   c.ShowFinalRanks(); // it will print the time each car needed to finish the circuit sorted from the fastest car to the   slowest.
   c.ShowWhoDidNotFinish(); // it is possible that some cars don't have enough fuel to finish the circuit
   return 0;
}
```

Organize the code in the following way:

- a header file for each different car (Dacia, Toyota, …), a header file for the abstract class **Car**, a header file for the Weather enum and one for **Circuit.**
- a .cpp file for each different implementation of a car (Dacia, Toyota, etc.). Abstract class **Car** only has virtual methods (it does not need a .cpp file).
- a .cpp file for **Circuit** class.
- a main file called **main.cpp** that contains the main function.

LAB-7 (TEMPLATES & LITERALS)

1. Write two C++ literals for Kelvin and Fahrenheit conversions to Celsius so that the following code works.

```
int main() {
    float a = 300_Kelvin;
    float b = 120_Fahrenheit;
    return 0;
}
```

Organize the code in the following way:

- a main file called **main.cpp** that contains the main function and has an example on how to use those two literals (similar to the explained in the previous main function) .
- Use the following **link** for explanation on how to convert from Kelvin and Fahrenheit to Celsius.

2. Write a template that simulates a vector template. This template should cover the following methods:

- **push** method (to add an element at the end of the vector)
- **pop** method (return the last element from the vector)
- **delete/remove** method (deletes/removes an element from a specific index in the vector)
- **insert** method (insert an element at a specific index)
- **sort** method (sort all elements in the vector ==> use a callback function to provide a way to compare two elements. If the callback function is not present (*nullptr*), **operator<** is used.)
- **get** method (returns a constant reference to an object from the vector)
- **set** method (set an object to a specific index --> that index must be already used by another object).
- **count** method (returns the number of elements in the vector)
- **firstIndexOf** method (given an object and a callback function, find the first index of the element from the vector that is equal (according to the callback function) with the given parameter. If the callback function is nor present (*nullptr*), **operator==** is used.

Use dynamic allocation. Whenever, resizing is needed, double the size of the allocated space.

LAB-8 (STL)

Write a program that does the following:

- Reads a phrase / sentence from a text file into a **std::string** object
- Split the phrase in words. Consider that the following characters are separators (space, comma, question and exclamation mark, point). Words may be separated by multiple spaces. Use only methods available in the **std::string** class for this task
- Create a **std::map** object that counts how many times each word is encountered. Don't take word casing into consideration (case-insensitive).
- Use a **std::priority_queue** to sort each word using the number of times it was found. if two words have the same number of appearance in the phrase, they will be sorted lexicographically.
- Print the words after they are sorted

Example:

- Let's assume that we have a file that contains the following phrase: "***I bought an apple. Then I eat an apple. Apple is my favorite.***"
- First we create a MAP with the following items:

```
{
     "i" : 2,
     "bought" : 1,
     "an" :  2,
      "apple" : 3,
      "then" : 1,
      "eat" : 1,
      "is" : 1,
      "my" : 1,
      "favorite" : 1
}
```

- Then we sort these words based on the amount of time we found them and lexycographically
- Finally - we will print the following:
  - `apple => 3`
  - `an => 2`
  - `i => 2`
  - `bought => 1`
  - `eat => 1`
  - `favorite => 1`
  - `is => 1`
  - `my => 1`
  - `then => 1`

LAB-9 (FOR-EACH & STRUCTURED BINDINGS)

Write a C++ template **Map** that will make the following code :

```
int main()
{
    Map<int, const char *> m;
    m[10] = "C++";
    m[20] = "test";
    m[30] = "Poo";
    for (auto[key, value, index] : m)
    {
        printf("Index:%d, Key=%d, Value=%s\n", index, key, value);
    }
    m[20] = "result";
    for (auto[key, value, index] : m)
    {
        printf("Index:%d, Key=%d, Value=%s\n", index, key, value);
    }
    return 0;
}
```

will print the following on the screen:

```
Index:0, Key=10, Value=C++
Index:1, Key=20, Value=test
Index:2, Key=30, Value=Poo
Index:0, Key=10, Value=C++
Index:1, Key=20, Value=result
Index:2, Key=30, Value=Poo
```

For this task, use the following:

- structured binding
- auto keyword
- for-each

Besides this - add the following methods:

- a method **Set** that can be used to associate a value to a key
- a method **Get** using the following syntax ***bool Get(const K& key, V& value)*** that will copy the value associated to parameter **key** into the parameter **value** and returns **true**. If the parameter **key** does not exists in the map, **Get** method will return **false**
- a method **Count** that returns the amount of elements in the map
- a method **Clear** that clears the entire map
- a method **Delete** that deletes a specific key (if exists) --> use the following syntax: *bool Delete(const K& key)*
- a method **Includes** with the following syntax: ***bool Includes(const Map<K,V>& map)*** that checks if a map is included in another. A map A is included in another map B, if all keys from map A exists in map B

YOU ARE NOT ALLOWED TO USE STL TEMPLATES (Vector, Map, List, etc) for this problem.

LAB-10 (EXCEPTIONS)

Below you will see an example on how to use exceptions:

```
#include <iostream>
#include <exception>
using namespace std;

class exceptie1 : public exception
{
        virtual const char* what() const throw()
        {
                return "Impartire la 0!";
        }
};

class exceptie2 : public exception
{
        virtual const char* what() const throw()
        {
                return "Indexul este inafara domeniului!";
        }
};

int main()
{
        int z=-1, x = 50, y = 0;
        int arr[4] = { 0 };
        int i = 10;
        exceptie1 divide0;
        exceptie2 index_out_of_bounds;

        try
        {
                if (y==0)
                        throw divide0;
                z = x / y;
                cout << "Fara exceptie: z=" << z << endl;
        }
        catch (exception& e)
        {
                cout << "Exceptie: " << e.what() << endl;
        }

        try
        {
                if (i > 3)
                        throw index_out_of_bounds;
                arr[i] = z;
                for (i = 0; i < 4; i++)
                        cout << "arr[" << i << "] = " << arr[i] << endl;
        }
        catch (exception& e)
        {
                cout << "Exceptie: " << e.what() << endl;
        }

        return 0;
}
```

Problem 1: Build an array template (similar to the one below) and add exceptions to it.

```
class Compare
{
public:
        virtual int CompareElements(void* e1, void* e2) = 0;
};
template<class T>
class ArrayIterator
{
private:
        int Current; // mai adaugati si alte date si functii necesare pentru iterator
public:
        ArrayIterator();
        ArrayIterator& operator ++ ();
        ArrayIterator& operator -- ();
        bool operator= (ArrayIterator<T> &);
        bool operator!=(ArrayIterator<T> &);
        T* GetElement();
};
template<class T>
class Array
{
private:
        T** List; // lista cu pointeri la obiecte de tipul T*
        int Capacity; // dimensiunea listei de pointeri
        int Size; // cate elemente sunt in lista
public:
        Array(); // Lista nu e alocata, Capacity si Size = 0
        ~Array(); // destructor
        Array(int capacity); // Lista e alocata cu 'capacity' elemente
        Array(const Array<T> &otherArray); // constructor de copiere

        T& operator[] (int index); // arunca exceptie daca index este out of range

        const Array<T>& operator+=(const T &newElem); // adauga un element de tipul T la sfarsitul listei si returneaza this
        const Array<T>& Insert(int index, const T &newElem); // adauga un element pe pozitia index, retureaza this. Daca index e invalid arunca o
exceptie
        const Array<T>& Insert(int index, const Array<T> otherArray); // adauga o lista pe pozitia index, retureaza this. Daca index e invalid arunca o
exceptie
        const Array<T>& Delete(int index); // sterge un element de pe pozitia index, returneaza this. Daca index e invalid arunca o exceptie

        bool operator=(const Array<T> &otherArray);

        void Sort(); // sorteaza folosind comparatia intre elementele din T
        void Sort(int(*compare)(const T&, const T&)); // sorteaza folosind o functie de comparatie
        void Sort(Compare *comparator); // sorteaza folosind un obiect de comparatie

        // functii de cautare - returneaza pozitia elementului sau -1 daca nu exista
        int BinarySearch(const T& elem); // cauta un element folosind binary search in Array
        int BinarySearch(const T& elem, int(*compare)(const T&, const T&));//  cauta un element folosind binary search si o functie de comparatie
        int BinarySearch(const T& elem, Compare *comparator);//  cauta un element folosind binary search si un comparator

        int Find(const T& elem); // cauta un element in Array
        int Find(const T& elem, int(*compare)(const T&, const T&));//  cauta un element folosind o functie de comparatie
        int Find(const T& elem, Compare *comparator);//  cauta un element folosind un comparator

        int GetSize();
        int GetCapacity();

        ArrayIterator<T> GetBeginIterator();
        ArrayIterator<T> GetEndIterator();
};
```

LAB-11 (TESTING)

In fiecare grupa, se vor face grupuri de câte 2 studenți. Dacă numărul de studenți este par, se va face și un grup de 3 studenți și restul de 2 studenți.

In cadrul fiecărui grup, se vor realiza următoarele task-uri:

1. Fiecare student scrie un algoritm de sortare templetizat și care primește ca și parametri o lista de elemente de tipul template-ului,
2. In cadrul acelui algoritm, se vor introduce voit un număr de 5 erori de logica / funcționare
3. După terminarea acestui algoritm, fiecare student va furniza codul colegului din grup. Dacă avem un grup de 3 persoane (A, B și C - atunci A da codul lui B, B da codul lui C și C da codul lui A)
4. Fiecare student scrie o funcție main care testează codul primit de la colegul din grup. Funcția main trebuie în urma execuției sa găsească cele 5 erori pe introduse void de coleg. **ATENȚIE** - erorile sunt secrete (cunoscute **DOAR** de cel care le-a introdus în cod) și nu trebuie comunicate - idea e sa simulam un sistem de testare care chiar sa identifice aceste erori !!!

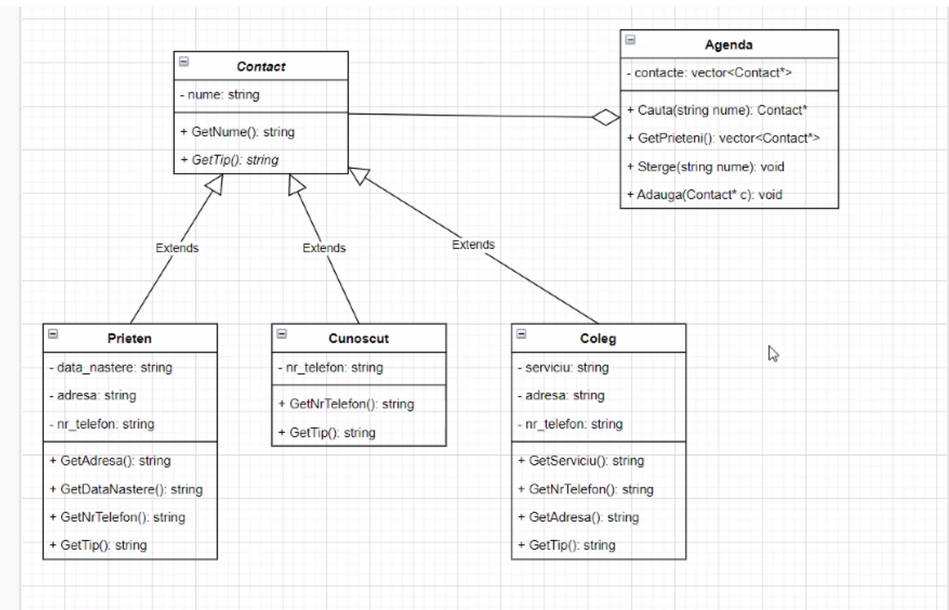
LAB-12 (OOP)

Problema 1: Creati urmatoarea ierarhie de clase.

- Clasa abstracta Contact care contine numele unei persoane
- Clasa Prieten derivata din clasa Contact care contine si data de nastere, numarul de telefon si adresa
- Clasa Cunoscut derivata din clasa Contact care contine numarul de telefon
- Clasa Coleg derivata din clasa Contact care contine numarul de telefon, firma la care lucreaza si adresa
- Clasa Agenda care contine mai multe elemente de tipul Contact. Clasa Agenta are urmatoarele functii
  - o functie pentru cautarea unui contact dupa nume
  - o functie care returneaza o lista de prieteni (contacte de tipul Prieten)
  - o functie care sterge un anumit contact dupa nume
  - o functie care adauga un anumit contact

Pentru crearea claselor se vor utiliza tehnici UML.

Pentru tipul de contact (Prieten, Cunoscut si Coleg) se va defini o enumerare (folosind tot metode UML) prin care se va descrie acest tip de date.

Creati si un cod in main.cpp care sa ilustreze cum functioneaza aceste componente.

Schema data la laborator: